Restaurant App MVP with Server and Database
Fall 2026 ? Assignment 2

1. Introduction

1.1 Purpose
This document defines the requirements of the Restaurant App MVP, its intended audience, and th
e version covered. Version 2 adds a Node.js/Express backend and MongoDB so every device reads a
nd writes the same data.

1.2 Scope
In-scope functionality includes: user signup and login with JWT, menu browsing/search/filter, c
art management, order placement and tracking, table reservation, and a manager dashboard. The E
xpress server, MongoDB database, REST APIs, and role-based protected routes are now in scope.

Out of scope: real payment gateways and push notifications.

Problems found in A1 (mock/local data) and how A2 solves them:

A1 problem	A2 solution
A manager price change on one phone did not appear on other phones.	Menu items live in Mong
oDB. GET /api/menu returns the current price to every device.
Marking a dish unavailable on one device left it orderable on others.	Availability is store
d on MenuItem.isAvailable. Place-order rejects unavailable items.
Reinstalling the app wiped orders and bookings.	Orders and reservations are saved on the se
rver, not only on the phone.
Login accounts existed only as a mock file inside the app.	Users are stored in MongoDB with
bcrypt-hashed passwords.
A customer order was invisible on a second phone.	GET /api/orders/my and GET /api/orders (m
anager) read the shared Orders collection.
Table bookings could collide because each phone kept its own list.	The server checks table
+ date + time before insert and returns 409 if taken.

1.3 Definitions and Acronyms
- **SRS**: Software Requirements Specification.
- **MVP**: Minimum Viable Product.
- **UML**: Unified Modeling Language.
- **Hook**: A React function that lets a component use state and lifecycle features.
- **Context**: A React API for sharing data across the component tree.
- **Reducer**: A pure function that takes previous state and an action and returns the next sta
te.
- **API**: Application Programming Interface; the HTTP contract between the app and the server.
- **REST**: Representational State Transfer; resources are accessed with HTTP methods.
- **Endpoint**: A specific URL and method, for example POST /api/auth/login.
- **JSON**: JavaScript Object Notation; the request and response data format.
- **MongoDB**: Document database used to store collections for the restaurant.
- **Collection**: A group of MongoDB documents, similar to a table.
- **Mongoose**: Node.js library that defines schemas and talks to MongoDB.
- **Middleware**: Express functions that run before a controller, such as JWT checks.
- **JWT**: JSON Web Token used after login to prove who the caller is.

2. Overall Description

2.1 User Roles
Role	Key permissions and goals
Customer	Browse menu, add items to a cart, place orders, track status, reserve tables, canc
el pending bookings.
Restaurant Manager	Edit menu (add, price, availability), view all orders, move order status
forward, accept or decline reservations.

2.2 User Stories
Unchanged from A1, except data now comes from the server: browse specials, book a table, filter
/search menu, add item notes, track order status, apply promo codes in the cart, and manager me
nu/order/reservation control.

3. Functional Requirements
Authentication:
- FR-01: The system shall allow users to toggle between Login and Signup.
- FR-02: The system shall validate email format and require a password with at least 8 characte
rs and one digit.
- FR-03: After login the app shall show customer tabs to customers and also the Dashboard to ma
nagers.

Menu & Search:
- FR-04: The system shall display at least 15 menu items across 4 categories from GET /api/menu
.
- FR-05: The system shall disable Add for unavailable items.
- FR-06: The header title shall show the count of currently displayed items.
- FR-07: Search input shall debounce after 400ms and call the menu search query.
- FR-08: The last 5 search terms shall appear as tappable suggestions.

Cart & Orders:
- FR-09: Users can increment, decrement, and remove cart items.
- FR-10: The cart shall show subtotal, 5% service, 15% tax, and an estimated grand total.
- FR-11: Promo codes WELCOME10 and FEAST20 apply to the client estimate only.
- FR-12: Customers choose Dine-in or Takeaway.
- FR-13: Order tracking shall poll GET /api/orders/my every 10 seconds (no fake timer). The ser
ver, not the client, owns status.

Reservation:
- FR-14: Hourly slots from 12:00 to 22:00.
- FR-15: Party size 1?12 and not above table capacity.
- FR-16: Pakistani mobile numbers 03XX-XXXXXXX.
- FR-17: Customers can cancel pending reservations.

Manager Dashboard:
- FR-18: Dashboard is visible only to the manager role.
- FR-19: Managers can accept or decline pending reservations.
- FR-20: Toggling availability and changing prices updates MongoDB so every customer sees the n
ew values after refresh.

4. Non-Functional Requirements
- Usability: Forms validate inline and clear errors when the user corrects a field.
- Performance: MenuItemCard uses React.memo; add-to-cart uses useCallback.
- Responsiveness: Screens use safe areas and work on phone-sized layouts.
- Maintainability: App code stays in src (api, screens, context, reducers, hooks, navigation).
Backend stays in server (config, models, routes, controllers, middleware).
- Data handling: Shared data is stored in MongoDB. The app stores only JWT and user profile in AsyncStorage so the session survives an app restart. Menu edits are not stored locally.
- Security: Passwords are hashed with bcrypt. Protected routes require Authorization: Bearer &l
t;token>. Menu writes require the manager role.

5. Database Design
Database name: ‘restaurant_app‘.

Collection	Fields and types	Relationships and rules
Users	name String required; email String required unique lowercase; password String require
d min 8; role enum customer|manager default customer	One user has many Orders and Reservatio
ns. Email is unique. Password is stored hashed, never returned by APIs.
MenuItems	name String required; description String; price Number required min 0; category S
tring required; isAvailable Boolean default true; imageUrl String; isSpecial Boolean default fa
lse	Orders store a copy of unit price plus a reference to MenuItem. Unavailable items cannot
be ordered.
Tables	tableNumber Number required unique; capacity Number required min 1; location String
Reservations reference a Table. Party size cannot exceed capacity.
Reservations	user ObjectId required ref User; table ObjectId required ref Table; reservatio
nDate Date required; timeSlot String required; partySize Number 1?12; phone String 03XX-XXXXXXX
; status enum Pending|Confirmed|Declined|Cancelled default Pending; createdAt/updatedAt	Un
ique pending/confirmed booking per table+date+time. Conflicting bookings return 409. Timestamps
enabled.
Orders	user ObjectId required ref User; items[{ menuItem ObjectId ref MenuItem, quantity mi
n 1, price min 0, note }]; totalAmount Number min 0; status enum Pending|Preparing|Ready|Ser
ved|Cancelled default Pending; type enum Dine-in|Takeaway; createdAt/updatedAt	totalAmount
is calculated on the server from database prices. Status may only move forward, or Cancelled wh
ile Pending. Timestamps enabled.

8. API Endpoints

Method	Endpoint	Access	Request body	Response
GET	/api/health	Public	None	status, time
POST	/api/auth/register	Public	name, email, password	message (201) or 409
POST	/api/auth/login	Public	email, password	token and user
GET	/api/menu	Public	None (query: category, search)	List of menu items
GET	/api/menu/:id	Public	None	One item or 404
POST	/api/menu	Manager	name, price, category, ...	Created item (201)
PUT	/api/menu/:id	Manager	fields to update	Updated item
DELETE	/api/menu/:id	Manager	None	Delete message
POST	/api/orders	Customer/Manager	items[{menuItem, quantity, note}], type	Created ord
er; total from DB
GET	/api/orders/my	Logged-in	None	Caller’s orders

| PATCH | /api/orders/:id/status | Manager | status | Updated order or 400 |
| GET | /api/reservations/tables | Public | None | List of tables |
| POST | /api/reservations | Logged-in | table, reservationDate, timeSlot, partySize, phone | C
reated reservation or 409 |
GET	/api/reservations/my	Logged-in	None	Caller’s reservations
GET	/api/reservations	Manager	None	All reservations
PATCH	/api/reservations/:id	Manager	status Confirmed|Declined	Updated reservation
PATCH	/api/reservations/:id/cancel	Owner	None	Cancelled reservation

9. Diagrams
See ‘A2/diagrams/sequence-order.svg‘ (App, Express Server, MongoDB) and ‘A2/diagrams/deployment
.svg‘ (phone, laptop server, MongoDB).